Stencils at the Roofline from Python: the Heat Equation and Jacobi Iteration on a Laptop GPU, and How Temporal Blocking Moves the Roof

Walid Hemati
Independent researcher

PROVISIONAL NUMBERS. The performance figures in this version come from `results/dev-provisional.json`, measured during development on a GPU shared with other jobs. They will be replaced by the serial, idle-GPU run of `bench/run.py -full`.

Abstract

The explicit heat equation with the 7-point (3D) or 5-point (2D) stencil, and the Jacobi iteration that shares its update, are the textbook memory-bound kernels: about one floating point operation per byte of DRAM traffic. We use them to test a simple claim: Python code on a GPU reaches the same *hardware efficiency* as hand-written CUDA C. One package, `pyheat3d`, implements the update in NumPy, PyTorch eager and `torch.compile`, Triton, NVIDIA Warp, Numba-CUDA and hand-written CUDA C compiled at run time through CuPy, all behind one interface and validated against analytic solutions (second order in space, first order in time, to the expected rates) and against a NumPy oracle. On an RTX 3070 Laptop GPU the best single-step kernel reaches 18.5 GLUPS on a 320^3 grid, which is 41% of the measured 361 GB/s copy bandwidth under a minimal 8-byte-per-update traffic model. We then add temporal blocking, which fuses k time steps into one DRAM pass. This does not beat the hardware. It lowers the traffic per update k-fold and so raises the per-step roofline by the same factor. With it the CUDA C variant reaches 22.5 GLUPS in 3D (0.50× the single-step roofline) and 120.1 GLUPS in 2D. We report both lattice updates per second and the estimated DRAM traffic behind them.

1 Introduction

Stencil codes sweep a grid and update each site from a few neighbours. They do very little arithmetic per byte moved, so on modern hardware their speed is set by memory bandwidth, not by floating point throughput. The roofline model [1] makes this exact: a kernel with arithmetic intensity I (FLOP/byte) cannot exceed $\min(P, I \cdot B)$, where P is peak compute and B peak bandwidth. For the explicit heat equation in single precision, $I \approx 1$ FLOP/byte and the bandwidth term dominates by more than an order of magnitude.

That makes the heat equation a fair test for high-level languages. There is no algorithmic headroom to hide behind and no tensor core to borrow. Whatever reaches the copy bandwidth has done the job, whether it is Python or C. This paper asks how close each mainstream Python GPU route gets, what it costs in code, and what happens when the algorithm itself changes the roof.

Contributions.

- An optimisation ladder for the same stencil in seven backends (NumPy, PyTorch eager, `torch.compile`, Triton, Warp, Numba-CUDA, CUDA C via CuPy/NVRTC), each as a variant in a common registry with one interface, one test suite and one benchmark harness.
- Validation that does not depend on any backend: the decaying Fourier mode against its continuous, semi-discrete and fully discrete exact solutions, Laplace and Poisson steady states, and 76/76 cross-backend comparisons against a NumPy oracle.
- A roofline analysis with a stated minimal traffic model. Every throughput is reported as lattice-site updates per second and as a percentage of the measured copy bandwidth (361 GB/s) and of the specification (448 GB/s).
- Temporal-blocking kernels written from Python (3D: shared memory plus warp shuffles with register blocking; 2D: warp-synchronous, no shared memory). They exceed the single-step roofline, and we explain exactly why this is not the same as exceeding the hardware.

2 Background and related work

Roofline. Williams, Waterman and Patterson [1] bound attainable performance by peak compute and by bandwidth times operational intensity, where intensity counts bytes of DRAM traffic. The bound depends on the *traffic*, not on the FLOPs alone. Any transformation that cuts traffic per unit of work raises the ceiling for that work.

Stencil optimisation. Datta et al. [2] auto-tuned 7-point stencils across multicore architectures and showed they are bandwidth bound once cache blocking removes redundant loads. On GPUs, Micikevicius [3] introduced the now standard 2.5D scheme: a thread block holds an xy tile in shared memory and marches along z, keeping the z neighbours in registers. Nguyen et al. [4] combined 2.5D spatial blocking with temporal blocking ("3.5D") on CPUs and GPUs. Holewinski, Pouchet and Sadayappan [5] generated GPU stencil code with overlapped tiling, recomputing halos redundantly instead of synchronising between blocks.

Temporal blocking. Wonnacott's time skewing [6] reorders the iteration space so that several time steps run on data still in cache. That removes the bandwidth limit in exchange for more work per byte. Wellein et al. [7] pipelined successive time steps across cores sharing a cache (wavefront parallelisation). Malas et al. [8] used diamond tiling with multicore wavefronts. Our temporally blocked kernels are a GPU form of the wavefront idea: a thread block streams through z and k pipeline stages advance the same columns by k time levels, using overlapped (redundant) halos as in [5].

Python on GPUs. NumPy [9] defines the array model the others imitate. PyTorch 2's `torch.compile` [10] captures Python into a graph and generates fused Triton kernels. Triton [11] lets one write block-level kernels in Python and leaves intra-block scheduling to the compiler. Warp [12] and Numba [13] compile SIMT kernels written in a Python subset. CuPy [14] provides NumPy-compatible arrays and runtime compilation of CUDA C through NVRTC. We use the last route as our "hand-written CUDA C" reference: the kernels are compiled at run time by NVRTC, so the package needs no CUDA toolkit or `nvcc` at install time.

3 Method

3.1 Physics and numerics

We solve $u_t = \alpha \nabla^2 u$ on the unit square or cube with forward Euler in time and second-order central differences in space (FTCS). With uniform spacing h and the mesh Fourier number $r = \alpha \Delta t / h^2$ the update is

$$u'_c = (1 - 2dr)\, u_c + r\, S_c, \qquad S_c = \big[(u_{x-1} + u_{x+1}) + (u_{y-1} + u_{y+1})\big] + (u_{z-1} + u_{z+1}), \tag{1}$$

where $d \in \{2, 3\}$. The von Neumann limit is $r \le 1/(2d)$, which the code enforces. Every backend uses the same bracketing of S_c, so they differ at most by fused multiply-add contraction. Dirichlet problems keep a fixed boundary layer that no kernel writes; periodic problems wrap indices. The Jacobi iteration for $\nabla^2 u = f$ is the same update with $c_0 = 0$, $c_1 = 1/(2d)$ and an additive term $-h^2 f/(2d)$. All kernels are written for the generic $u' = c_0 u + c_1 S$ and run both modes.

3.2 Reference solutions

For the mode $u_0 = \prod_i \sin(\kappa x_i)$ ($\kappa = \pi$ Dirichlet, 2π periodic) three exact references exist: (i) the PDE solution $e^{-\alpha d \kappa^2 t} u_0$; (ii) the semi-discrete (exact-in-time) solution $e^{-\alpha \lambda_h t} u_0$ with $\lambda_h = 4d \sin^2(\kappa h/2)/h^2$; and (iii) the fully discrete solution $g^n u_0$ with $g = 1 - 4dr \sin^2(\kappa h/2)$, which FTCS must reproduce to round-off. Error against (i) at fixed r measures the combined $\mathcal{O}(h^2) + \mathcal{O}(\Delta t) = \mathcal{O}(h^2)$ error. Error against (ii) on a fixed grid isolates the first-order temporal error. Error against (iii) is a pure implementation check. We avoid $r = 1/(6d)$, where the leading space and time error terms cancel.

For Laplace we use $\sin(\pi x)\sinh(\pi y)/\sinh \pi$ (2D) and $\sin(\pi x)\sin(\pi y)\sinh(\sqrt{2}\pi z)/\sinh(\sqrt{2}\pi)$ (3D), and quadratic harmonic functions on which the discrete Laplacian is exact. For Poisson we use a manufactured solution $u = \prod \sin(\pi x_i)$.

4 Implementation

4.1 One interface, a registry of variants

Each variant is a `Stepper` that owns two device buffers and exposes `step(n)`, `get()` and `synchronize()`. A registry entry records backend, dimensionality, storage type and k, the number of time steps fused per DRAM pass. Tests, validation and benchmarks loop over the registry. Variants whose package or GPU is missing are skipped. Table 1 lists the ladder.

4.2 Traffic model and roofline

We use the *minimal* traffic model: a sweep that advances the field by k steps must read each value once and write it once, so the DRAM traffic per lattice-site update is

$$B_{\text{upd}} = \frac{2\,b}{k}, \qquad b = \text{bytes per stored value.} \tag{2}$$

That is 8 B for fp32 single-step kernels and 4 B for fp16/bf16 storage. Reported effective bandwidth is LUPS $\times\, B_{\text{upd}}$. Halo re-reads and cache misses are not counted, so this is a lower bound on true traffic; an inefficient kernel cannot inflate it. With the measured 361 GB/s copy bandwidth the single-step fp32 roofline is $361/8 = 45.1$ GLUPS and the 2-byte roofline is 90.2 GLUPS. The update needs $2d + 2$ FLOPs (eq. (1)), so $I = 1$ FLOP/byte in 3D: at least 25× below the ridge point of this GPU.

Table 1: The optimisation ladder. k: time steps per DRAM pass.

backend	variant	k	idea
numpy	slicing	1	vectorised slices, CPU; correctness oracle
torch-eager	slicing	1	same slices on the GPU; one kernel per operator
torch-compile	inductor	1	the same Python traced and fused into one Triton kernel
triton	tile(-fp16/-bf16)	1	2D/3D tiles, neighbours as shifted masked loads
triton	zmarch(-fp16/-bf16)	1	3D: xy tile marches in z with a register queue
warp, numba	naive, zmarch	1	one thread per site; per-thread z column with register queue
cupy-cuda-c	naive(-fp16/-bf16)	1	one thread per site (several per thread for 2-byte types)
cupy-cuda-c	zmarch, ymarch	1	2.5D: shared-memory plane + register queue [3]
cupy-cuda-c	tb2, tb3, tb4, tb8	k	temporal blocking (wavefront, overlapped halos)
cupy-cuda-c	tb4-fp16	4	temporal blocking with fp16 storage

What temporal blocking does to the roof. For a k-step kernel eq. (2) gives $8/k$ bytes per fp32 update, so its roofline is $k \cdot 45.1$ GLUPS. A temporally blocked kernel that delivers more than 45.1 GLUPS has not moved data faster than the hardware allows. It moves k times less data per update. For these kernels we therefore report three numbers: the update rate; the DRAM traffic implied by eq. (2) (always below the copy bandwidth); and the "single-step-equivalent bandwidth" LUPS $\times$ 8 B. The last one is *not* a DRAM rate, only the bandwidth a $k = 1$ kernel would need to match the rate. For the CUDA C kernels we also give a more realistic *halo model* that adds the redundant halo reads of overlapped tiling (section 4.3), assuming no L2 reuse between blocks.

4.3 Kernels

Tiles, compiled expressions and naive SIMT. The Triton tile kernels load the centre tile and its four or six shifted neighbours as masked loads. Reuse between neighbouring loads is left to L1/L2. `torch.compile` receives the NumPy-style expression of eq. (1) with slicing (Dirichlet) or `torch.roll` (periodic) and lowers it to one fused kernel. Warp, Numba and the naive CUDA C kernels use one thread per site. For 2-byte storage the CUDA C kernel handles several sites per thread, to keep enough bytes in flight.

2.5D z-marching. Following [3], the CUDA C `zmarch` kernel stages the current z plane plus a one-site halo in shared memory and keeps the $z \pm 1$ values in a register queue. It prefetches the next plane one iteration ahead. The Triton, Warp and Numba `zmarch` variants keep the register queue and leave x/y reuse to the caches.

Temporal blocking. The 3D kernel assigns a $32 \times T_y$ thread block to an extended xy tile of $32 \times (T_y R)$ columns. Lane x owns a column and each thread owns R consecutive y rows (register blocking). The block streams through the planes of a z chunk. At iteration i_z it loads level t of plane i_z, and stage $s = 1..k$ computes level $t + s$ of plane $i_z - s$ from a register queue of level $t + s - 1$. That is a wavefront in (z, t), as in [7], mapped onto one thread block. The x neighbours come from warp shuffles, the y neighbours from the thread's own registers, and only the first and last row of each thread pass through shared memory, with one barrier per stage. The halo is recomputed redundantly (overlapped tiling [5]). The valid region shrinks by one column per stage,

and only the central $(32 - 2k) \times (T_y R - 2k)$ columns are stored, once per k steps. The 2D kernel is warp-synchronous: each warp owns an x strip of $32R$ columns, with lane l holding columns $l, l + 32, \ldots$ so every load is coalesced. It marches along y with no shared memory and no barriers. The chunk length along the marching direction is chosen at run time so small grids still fill the GPU. Intermediate levels never leave registers, so the fp32 kernels give the same arithmetic as k single steps and the fp16 kernel rounds to storage only once per k steps.

All CUDA C kernels are one source file compiled by NVRTC with macros for storage type, boundary condition, k and tile shape. The Python side picks the launch configuration. The backends together are about 1,400 lines of Python and CUDA C, comments included.

5 Validation

Figure 1 summarises the studies. They run with the fp64-capable variant `torch-eager/slicing`, except panel (d). Against the PDE solution at fixed $r = 0.1$ the observed spatial order is 1.97 (2D) and 1.99 (3D) with Dirichlet walls, and 2.00/2.08 on periodic grids. The finest 3D grid (129^3) has relative error 2.4×10^{-5}. Against the semi-discrete solution on a fixed grid the temporal order is 1.00 (2D, $n = 65$) and 1.00 (3D, $n = 33$). The converged Jacobi solutions converge to the smooth harmonic functions with order 1.99 (2D) and 1.98 (3D). The manufactured Poisson problems converge with 2.00 and 2.01. On the quadratic harmonic function, where the stencil is exact, Jacobi recovers the solution to 5.3×10^{-15} (3D), which is round-off and iteration tolerance only.

Every GPU variant reproduces the fully discrete FTCS solution of the Fourier mode to at most 1.2×10^{-5} in 3D (fp32, 65^3, 410 steps), while its error against the PDE is 6.0×10^{-5}. Round-off is therefore far below discretisation error. On random fields with both boundary conditions, 76 of 76 comparisons of 23 variants against the NumPy oracle pass. The worst fp32 relative deviation is 5.1×10^{-7}, which comes from FMA contraction only. fp16 and bf16 variants are compared with a NumPy reference that applies the same storage rounding after every DRAM pass (worst 8.9×10^{-4} and 7.1×10^{-3}). That makes the test exact up to a few storage ulps.

Storage precision matters for physics: on the decaying mode the fp16 and bf16 single-step variants reach 7.3×10^{-2} and 3.5×10^{-1} against the PDE in 3D, against 6.0×10^{-5} in fp32, because each step's small increment $r\nabla^2 u$ is rounded to an 8- or 11-bit mantissa. Temporal blocking with fp16 storage rounds only every fourth step and lands at 7.8×10^{-4}. Low-precision storage halves the roofline cost, but for long diffusive runs bf16 is not accurate enough.

6 Performance

All numbers come from `bench/run.py` (`results/dev-provisional.json`, 184 runs, NVIDIA GeForce RTX 3070 Laptop GPU, driver 595.91.07, PyTorch 2.7.0+cu126, Triton 3.3.0, Warp 1.18.0, CuPy 13.5.1, Numba 0.68.0, Python 3.13.2). Each run warms up (including JIT compilation), synchronises the device around the timed region and reports the median of five repeats, each long enough to be timed reliably, together with min and max. Dirichlet heat mode at $r = 0.9/(2d)$, fp32 unless stated.

Single step: Python at the roof. In 3D (320^3) the best single-step variant, `triton/zmarch`, runs at 18.5 GLUPS (min–max 18.5–18.7). That implies 148 GB/s of DRAM traffic: 41% of the measured copy bandwidth and 33% of the specification. In 2D (8192^2) the best is `cupy-cuda-c/ymarch` with 32.7 GLUPS (72% of copy). Figure 2 and table 2 show that several Python routes (Triton,

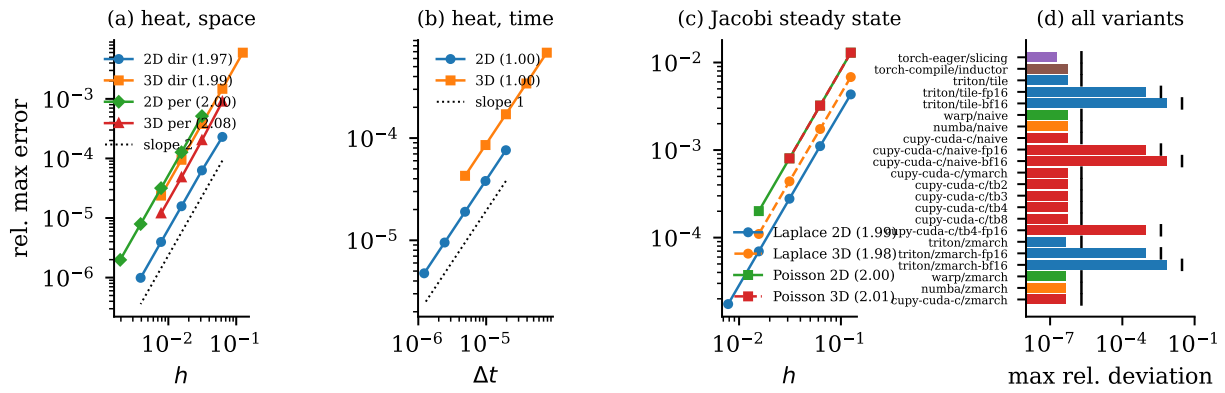


Figure 1: Validation. (a) Error against the exact PDE solution of a decaying sine mode at fixed $r = 0.1$ (2nd order). (b) Error against the semi-discrete solution on a fixed grid (1st order in Δt). (c) Jacobi steady states for Laplace and Poisson. (d) Worst relative deviation of every GPU variant from the (storage-emulating) NumPy reference; ticks mark the tolerance. Observed orders in the legends.

`torch.compile`, and the Warp/Numba z-marching kernels in 3D) land near the same ceiling as the hand-written CUDA C. They are separated more by their small-size launch overhead (fig. 4) than by their large-size bandwidth. PyTorch eager is far below, for a structural reason: each of its roughly ten operators is a separate pass over memory. The same Python expression handed to `torch.compile` runs as one kernel. Against NumPy on the 16-core CPU the best GPU kernel is 71× faster in 3D at 64^3.

Low-precision storage. fp16/bf16 storage halves B_{upd}. The best 2-byte single-step variant in 3D, `triton/zmarch-fp16`, reaches 36.9 GLUPS, that is 148 GB/s or 41% of copy bandwidth. 2-byte kernels need more independent loads in flight per thread to saturate DRAM, so they sit further below their own roof than fp32 kernels do below theirs.

Temporal blocking: a higher roof, not faster DRAM. The CUDA C kernel `cupy-cuda-c/tb3` ($k = 3$) delivers 22.5 GLUPS in 3D, 0.50× the single-step fp32 roofline. Under the minimal model it moves 60 GB/s (17% of copy). With the halo over-fetch of overlapped tiling the estimate is 80 GB/s (3.54 B per update). A single-step kernel would need 180 GB/s to match the rate, which no DRAM on this GPU provides. In 2D, `cupy-cuda-c/tb8` ($k = 8$) reaches 120.1 GLUPS (2.66× the single-step roof) while using 120 GB/s. The gain saturates with k: redundant halo work grows as $(T - 2k)^{-1}$ per tiled dimension, the pipeline holds $3k$ values per site in registers, and in 3D every stage costs a barrier. At large k these kernels become instruction-bound rather than bandwidth-bound. Figure 3 shows the same data on the classic roofline. Temporal blocking moves points to the right (higher intensity); it does not move them above the memory roof.

7 Discussion

Why Python gets there. A memory-bound kernel needs three things: coalesced access, enough bytes in flight, and no redundant passes. Triton's block model gives the first two by construction. `torch.compile` gives the third by fusion. Warp and Numba expose plain SIMT, so they reach the roof as soon as the indexing is coalesced, and their per-launch overhead is visible only on small grids.

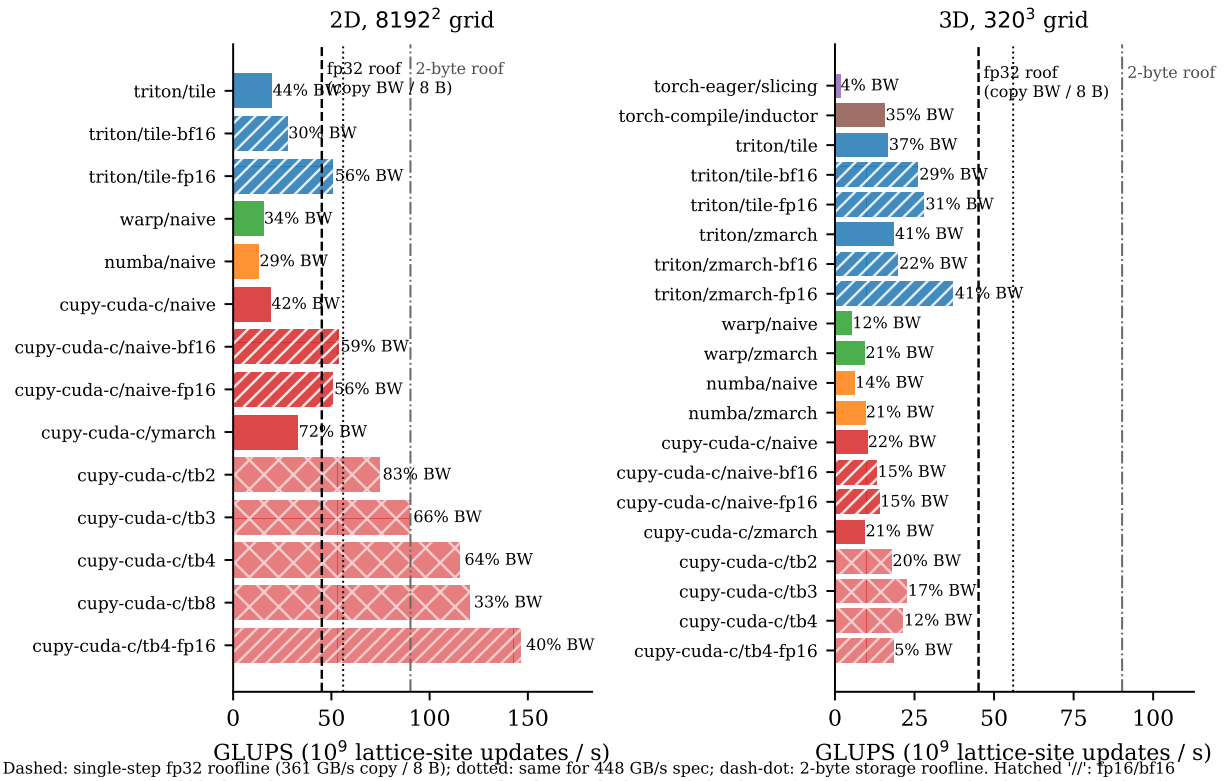


Figure 2: Throughput of every variant at the largest benchmarked size, against the fp32 single-step roofline (dashed, copy bandwidth / 8 B), the same at the specification bandwidth (dotted) and the 2-byte roofline (dash-dot). Labels give the share of copy bandwidth used under the minimal traffic model. Temporally blocked variants ("xx") sit above the single-step roof while using less bandwidth, because their roof is k times higher.

The interpreter costs microseconds per launch and nothing per byte. What Python does *not* give for free are the explicit shared-memory and warp-shuffle schedules that temporal blocking needs. Triton deliberately hides them. For that rung we wrote CUDA C, but compiled and launched it from Python.

Threats to validity. (i) One laptop GPU: absolute numbers do not transfer, but percentages of the measured copy bandwidth should transfer better. (ii) Laptop GPUs throttle with temperature and power. Each run records SM clock, temperature and power, and the final benchmark runs serially on an idle GPU. (iii) The numbers in this version were measured while other jobs shared the GPU and are pessimistic. (iv) The CUDA C reference is compiled by NVRTC, not `nvcc`. Both use the same NVVM and ptxas pipeline, but default flags differ. (v) The minimal traffic model ignores halo re-reads, and on small grids part of the working set stays in the 4 MB L2. Effective bandwidths for small grids can therefore exceed what DRAM alone could deliver. We base claims on the largest grids. (vi) Tile shapes were tuned briefly on a shared GPU and are unlikely to be optimal.

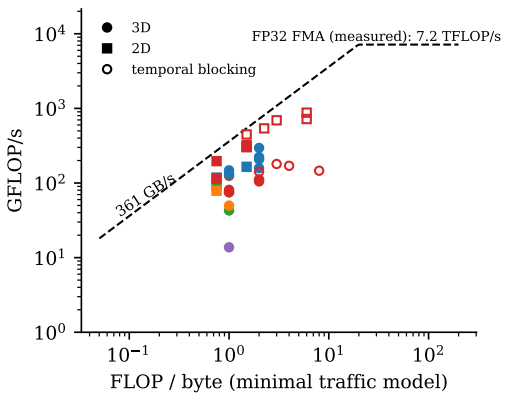


Figure 3: Classic roofline. Intensity uses the minimal traffic model, so temporally blocked kernels (open symbols) sit k times further right.

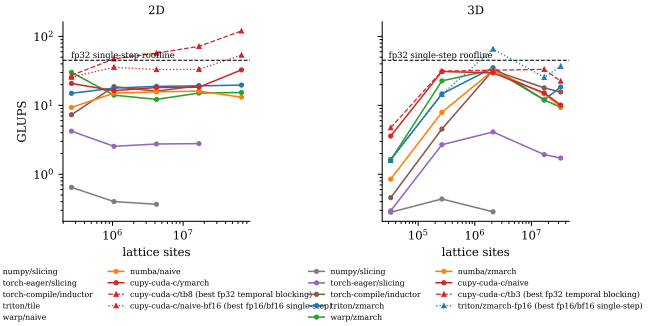


Figure 4: Throughput against problem size for the best fp32 single-step variant of each backend, the best temporally blocked and the best 2-byte variant. Small grids are bound by Python launch overhead and fit in the 4 MB L2.

8 Conclusion

For the canonical memory-bound stencil, Python reaches the hardware ceiling: Triton, `torch.compile`, Warp and Numba kernels run near the copy bandwidth, as hand-written CUDA C does. Beyond that ceiling only algorithmic change helps. Temporal blocking raises the update rate above the single-step roofline by cutting traffic per update. We report it as a change of roofline, with its traffic, not as a victory over the hardware.

Code availability

Code, benchmark JSON, figure scripts and this paper: https://github.com/walidhemati/pyheat3d (MIT licence). `bench/run.py -full` regenerates every number, and `figures/make_figures.py` regenerates every figure and the macro file this paper is typeset from.

References

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[2] Kaushik Datta, Mark Murphy, Vasily Volkov, Samuel Williams, Jonathan Carter, Leonid Oliker, David Patterson, John Shalf, and Katherine Yelick. Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing (SC08)*, pages 1–12, 2008. doi: 10.1109/SC.2008.5222004.

[3] Paulius Micikevicius. 3D finite difference computation on GPUs using CUDA. In *Proceedings of the 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, pages 79–84, 2009. doi: 10.1145/1513895.1513905.

Table 2: All variants at the largest benchmarked size. GB/s and % use the minimal traffic model $B_{\mathrm{upd}} = 2b/k$. "equiv.": single-step-equivalent bandwidth LUPS × 8 B (not a DRAM rate).

	backend	variant	store	k	GLUPS	B/upd	GB/s	% copy	% spec	equiv.
2D	`triton`	`tile`	fp32	1	19.69	8	158	44	35	–
2D	`triton`	`tile-bf16`	bfp16	1	27.50	4	110	30	25	–
2D	`triton`	`tile-fp16`	fp16	1	50.74	4	203	56	45	–
2D	`warp`	`naive`	fp32	1	15.36	8	123	34	27	–
2D	`numba`	`naive`	fp32	1	13.14	8	105	29	23	–
2D	`cupy-cuda-c`	`naive`	fp32	1	19.10	8	153	42	34	–
2D	`cupy-cuda-c`	`naive-bf16`	bfp16	1	53.52	4	214	59	48	–
2D	`cupy-cuda-c`	`naive-fp16`	fp16	1	50.43	4	202	56	45	–
2D	`cupy-cuda-c`	`ymarch`	fp32	1	32.72	8	262	72	58	–
2D	`cupy-cuda-c`	`tb2`	fp32	2	74.60	4	298	83	67	597
2D	`cupy-cuda-c`	`tb3`	fp32	3	89.94	2.7	240	66	54	720
2D	`cupy-cuda-c`	`tb4`	fp32	4	115.42	2	231	64	52	923
2D	`cupy-cuda-c`	`tb8`	fp32	8	120.13	1	120	33	27	961
2D	`cupy-cuda-c`	`tb4-fp16`	fp16	4	146.16	1	146	40	33	585
3D	`torch-eager`	`slicing`	fp32	1	1.72	8	14	4	3	–
3D	`torch-compile`	`inductor`	fp32	1	15.60	8	125	35	28	–
3D	`triton`	`tile`	fp32	1	16.63	8	133	37	30	–
3D	`triton`	`tile-bf16`	bfp16	1	25.97	4	104	29	23	–
3D	`triton`	`tile-fp16`	fp16	1	27.83	4	111	31	25	–
3D	`triton`	`zmarch`	fp32	1	18.48	8	148	41	33	–
3D	`triton`	`zmarch-bf16`	bfp16	1	19.76	4	79	22	18	–
3D	`triton`	`zmarch-fp16`	fp16	1	36.95	4	148	41	33	–
3D	`warp`	`naive`	fp32	1	5.36	8	43	12	10	–
3D	`warp`	`zmarch`	fp32	1	9.39	8	75	21	17	–
3D	`numba`	`naive`	fp32	1	6.23	8	50	14	11	–
3D	`numba`	`zmarch`	fp32	1	9.50	8	76	21	17	–
3D	`cupy-cuda-c`	`naive`	fp32	1	10.12	8	81	22	18	–
3D	`cupy-cuda-c`	`naive-bf16`	bfp16	1	13.14	4	53	15	12	–
3D	`cupy-cuda-c`	`naive-fp16`	fp16	1	13.94	4	56	15	12	–
3D	`cupy-cuda-c`	`zmarch`	fp32	1	9.43	8	75	21	17	–
3D	`cupy-cuda-c`	`tb2`	fp32	2	17.72	4	71	20	16	142
3D	`cupy-cuda-c`	`tb3`	fp32	3	22.48	2.7	60	17	13	180
3D	`cupy-cuda-c`	`tb4`	fp32	4	21.24	2	42	12	9	170
3D	`cupy-cuda-c`	`tb4-fp16`	fp16	4	18.26	1	18	5	4	73

[4] Anthony Nguyen, Nadathur Satish, Jatin Chhugani, Changkyu Kim, and Pradeep Dubey. 3.5-D blocking optimization for stencil computations on modern CPUs and GPUs. In *2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis (SC10)*, pages 1–13, 2010. doi: 10.1109/SC.2010.2.

[5] Justin Holewinski, Louis-Noël Pouchet, and P. Sadayappan. High-performance code generation for stencil computations on GPU architectures. In *Proceedings of the 26th ACM International Conference on Supercomputing (ICS '12)*, pages 311–320, 2012. doi: 10.1145/2304576.2304619.

[6] David Wonnacott. Using time skewing to eliminate idle time due to memory bandwidth and network limitations. In *Proceedings of the 14th International Parallel and Distributed Processing Symposium (IPDPS 2000)*, pages 171–180, 2000. doi: 10.1109/IPDPS.2000.845979.

[7] Gerhard Wellein, Georg Hager, Thomas Zeiser, Markus Wittmann, and Holger Fehske. Efficient temporal blocking for stencil computations by multicore-aware wavefront parallelization. In *2009 33rd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*, pages 579–586, 2009. doi: 10.1109/COMPSAC.2009.82.

[8] Tareq Malas, Georg Hager, Hatem Ltaief, Holger Stengel, Gerhard Wellein, and David Keyes.

Multicore-optimized wavefront diamond blocking for optimizing stencil updates. *SIAM Journal on Scientific Computing*, 37(4):C439–C464, 2015. doi: 10.1137/140991133.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[10] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 929–947, 2024. doi: 10.1145/3620665.3640366.

[11] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

[12] Miles Macklin. Warp: A high-performance python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), https://github.com/nvidia/warp, March 2022.

[13] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.

[14] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.